

Algorithm: An algorithm is a finite sequence of instructions each of which has a clear meaning and can be performed with a finite amount of effort in a finite length of time. No matter what the input values may be, an algorithm terminates after executing a finite number of instructions. In addition, every algorithm must satisfy the following criterias: They are

- 1) Input: There are zero or more quantities, which are externally supplied.
- 2) Output: at least one quantity is produced.
- 3) Definiteness: each instruction must be clear & unambiguous.
- 4) Finiteness: if we trace out the instructions of an algorithm, then for all cases the algorithm will terminate after a finite number of steps.
- 5) Effectiveness: - Every instruction should be fundamental to be carried out, in principle by a person or person using only pen and paper.

Performance Analysis: - The performance of a program is the amount of computer memory and time needed to

Given a program, we can analyse its performance of an algorithm in two ways they are 1) Time complexity 2) Space complexity

1) Time complexity: The amount of time that an algorithm requires for its execution is known as time complexity. There are mainly 3 types of time complexity's available. They are

- a) Best Case Time Complexity
- b) worst case " " "
- c) Average case " " "

Best Case Time Complexity :- If an algorithm requires minimum amount of time for its execution.

worst case Time Complexity :- if an algorithm requires maximum amount of time for its execution.

Average case Time complexity :- if an algorithm requires average amount of time for its execution.

Example :- Let's take Linear Search.

10 20 30 40 50

key = 40

Compare key element with elements of the array one by one until key element found.

$10 \neq 40$, $20 \neq 40$, $30 \neq 40$, $40 = 40$ key found

→ if the key element found at first position then it's Best Case

→ if the key element found at last position then it is called worst case Time complexity why because worst case means algorithm requires maximum of time.

→ if the key element found at 2nd or 3rd position then it is average case Time complexity.

In order to calculate time complexity, we have two approaches

1) Frequency count (or) step count

2) Asymptotic Notations.

2) Space Complexity :- The amount space that algorithm requires for its execution is known as space complexity.

Frequency Count :- It specifies the number of times a statement is to be executed. How many times the corresponding statement is executed that is called frequency count.

1. For ~~comment~~; //comment ;
 declaration Statement; int a;
 count is 0 (zero)

because comments are executed.

2. return Statement; return 0; → 1
 Assignment Statement; a=1; → 1
 count is '1'.

3. Ignore lower order exponents, when higher order exponents are present.

$$\text{eg: } 3n^4 + 4n^3 + 10n^2 + n + 100$$

ignore

$$= 3n^4$$

$$= O(n^4) \Rightarrow \text{order of } n^4.$$

4. Ignore constant multiple.

eg:- int sum(int a[], int n)

{ s=0; // assignment s and step count = 1

for(i=0; i<n; i++) → n+1

{ s = s + a[i]; → n

} return s; → 1

}

$$2n+3$$

ignore

Let n=3

0 < 3 → 1

1 < 3 → 2

2 < 3 → 3

3 < 3 → false 4.

→ This condition is checked for n+1 times.

→ body is executed only 3 times.

∴ time complexity is $O(n)$.

↳ order of.

Space Complexity:- we need to calculate the space of each element

eg:- int sum(int a[], int n)

{ s=0;

for(i=0; i<n; i++)

{ s = s + a[i];

return S;

}

$a \rightarrow n$ $\therefore a$ occupies n elements (or) words.
 $\downarrow$
 is array

$n \rightarrow 1$ $\therefore$ no. of elements.

$S \rightarrow 1$ $\therefore$ it occupies one element

$i \rightarrow 1$ $\therefore$ it is a variable.

5	6	9
---	---	---

 $n+3$

Space complexity is $O(n)$
 Big O of n .

another example for frequency count (or) step count:-

eg:- void mat_add(int a[][], int b[][])

{

int c[][];

for($i=0; i<n; i++$) $\rightarrow n+1$

{
 for($j=0; j<n; j++$) $\rightarrow n \times (n+1)$

{

$c[i][j] = a[i][j] + b[i][j]; \rightarrow n \text{ times, } n^2$
 (n x n)

}

}

$\Rightarrow$ so let $n=3$, 3×3 matrix

-	-	-
-	-	-
-	-	-

 $\Rightarrow n+1$ $\Rightarrow n \times (n+1) \Rightarrow n^2 + n$ $\Rightarrow n \times n = n^2$

From $2n^2 + 2n + 1$
 ignore

$\therefore$ time complexity is Big $O(n^2)$.

Space complexity is

$$a \rightarrow n^2 \quad [n \times n, 3 \times 3 = 9]$$

$$b \rightarrow n^2$$

$$c \rightarrow n^2$$

$$i \rightarrow 1$$

$$j \rightarrow 1$$

$$n \rightarrow 1$$

$$\underline{3n^2 + 3}$$

ignore

ignore

$$\Rightarrow n^2$$

Space complexity is $O(n^2)$.

another example:-

void mat_mul(int a[][], int b[][])

{

for(i=0; i<n; i++) $\rightarrow n+1$

{

for(j=0; j<n; j++) $\rightarrow n \times (n+1)$

{

c[i][j]=0; $\rightarrow n \times n$

for(k=0; k<n; k++) $\rightarrow n \times n \times (n+1)$

{

c[i][j] = c[i][j] + a[i][k] * b[k][j]; $\rightarrow n;$

}

}

}

$$\Rightarrow n+1$$

$$n^2 + n$$

$$n^2$$

$$n^3 + n^2$$

$$n^3$$

time complexity is $O(n^3)$

↓
order

$$\Rightarrow 2n^3 + 3n^2 + 2n + 1$$

Space complexity :-

$$a \rightarrow n^2$$

$$b \rightarrow n^2$$

$$c \rightarrow n^2$$

$$n \rightarrow 1$$

$$i \rightarrow 1$$

$$j \rightarrow 1$$

$$k \rightarrow 1$$

$$\Rightarrow 3n^2 + 4$$

Space complexity is $O(n^2)$

2) Asymptotic notations :- We have to calculate time complexity and space complexity, they are 5 type to calculate it. They are

1) Big oh notation (O)

2) Big omega notation (Ω)

3) Theta notation (Θ)

4) Little oh notation (o)

5) Little omega notation (ω)

Big oh notation (O) :- (worst case)

The 2 differentiable functions $f(n)$ & $g(n)$

$f(n)$ grows with same rate (or) slower than $g(n)$

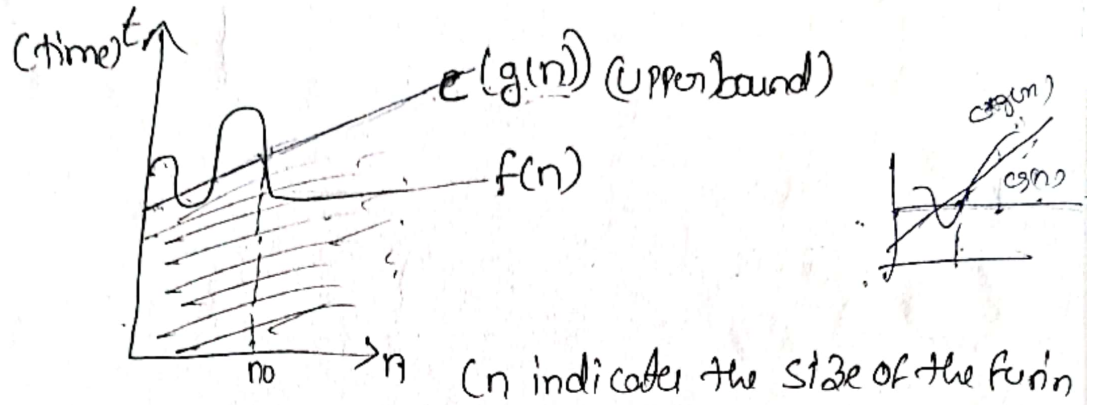
$$f(n) \leq c * g(n), \quad n \geq n_0, \quad c > 0, \quad n_0 \geq 1$$

$$f(n) = O(g(n))$$

logically broken down into

$$f(n) = \Theta(g(n)) \quad \text{or} \quad f(n) = o(g(n))$$

(Same rate) (slower)



$g(n)$ is an asymptotic upper bound for $f(n)$

Eg:- $f(n) = 3n+2$; $g(n) = n$

$$f(n) = O(g(n))$$

Big Oh.

$$f(n) \leq C * g(n) \quad n > 0, n_0 \geq 1$$

$$3n+2 \leq C * n \quad \text{Assume } C=4$$

$$3n+2 \leq 4n$$

$$n \geq 2$$

$$3(2)+2 \leq 4(2)$$

$$6+2 \leq 8 \quad \therefore 8 \leq 8$$

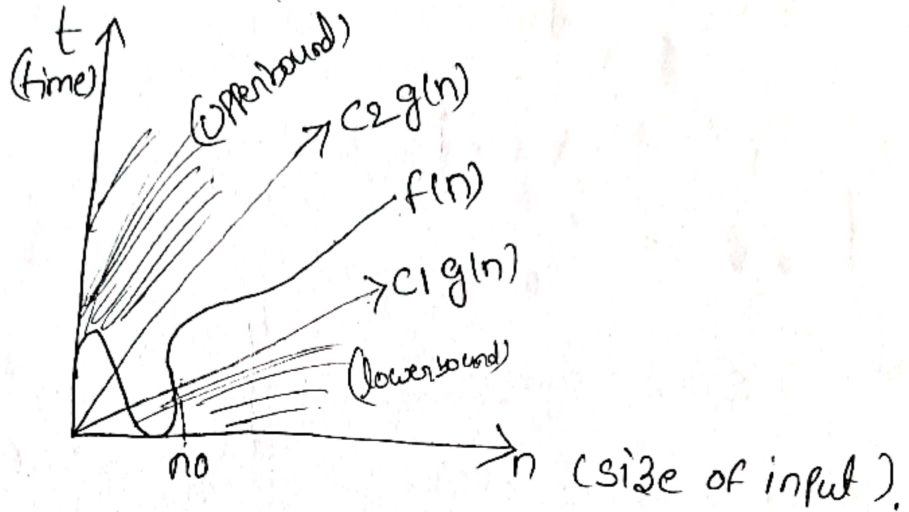
2) The ta Θ -notation :- (Equality) (Average case)

we choose $f(n)$ & $g(n)$ as two differentiable functions, &

Say that they have same growth rate.

$$\text{if, } \lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = c \quad : 0 < c < \infty$$

formally stated as $f(n) = O(g(n))$.



$$c_1 * g(n) \leq f(n) \leq c_2 * g(n)$$

$$c_1, c_2 > 0$$

$$n \geq n_0, n_0 \geq 1$$

eg:- $f(n) = 3n+2$
 $g(n) = n$

$$c_1 * g(n) \leq f(n) \leq c_2 * g(n)$$

↓

$$f(n) \leq c_2 * g(n)$$

$$3n+2 \leq 4(n) \quad (c_2=4)$$

$$n_0 \geq 2$$

Take $n=2$

$$3(2)+2 \leq 4(2)$$

$$8 \leq 8$$

~~$n_0 \geq 1$~~ $n_0 \geq 2$

$$f(n) \geq c_1 * g(n)$$

$$3n+2 \geq 1 * n$$

($c_1=1$ lower bound)

$$3n+2 \geq n$$

$$3(2)+2 \geq 2$$

$$8 \geq 2$$

3) Big - omega Notation (Ω):- (Best case)

The 2 differentiable functions $f(n)$ & $g(n)$

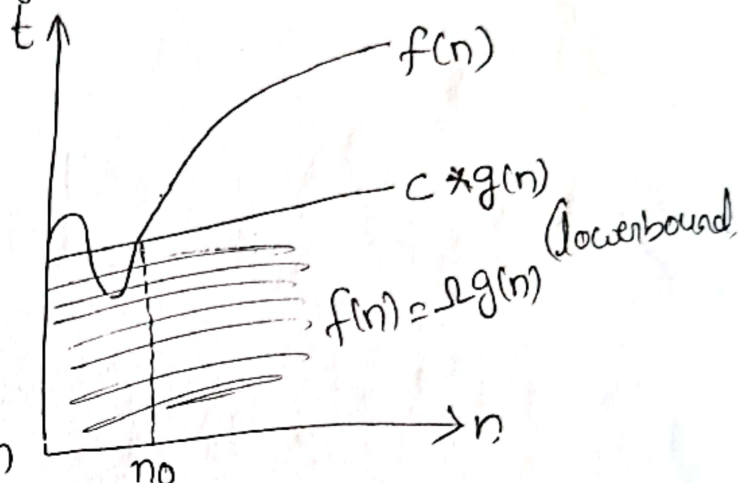
$f(n)$ grows with same rate (or) faster than $g(n)$

$$f(n) \geq c * g(n) \quad ; \quad n \geq n_0, \quad c > 0, \quad n \geq 1$$

denoted by $f(n) = \Omega(g(n))$

→ $g(n)$ is an asymptotic lower bound of $f(n)$

eg:- $f(n) = 3n+2, g(n) = n$



$$f(n) \geq c * g(n)$$

$$3n+2 \geq cn \quad \therefore c = 1$$

$$\boxed{3n+2 \geq n} \quad n_0 \geq 1$$

$$3n+2 = \Omega(n)$$

$$3(1)+2 \geq 1$$

$$5 \geq 1 \quad \text{true}$$

$$f(n) \geq c * g(n)$$

$$\boxed{3n+2 = \Omega(n)}$$

④ little oh:- little oh - slower growth rate.

$f(n)$ grows slower than $g(n)$

$$\text{if } \lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = 0$$

formally stated as $f(n) = o(g(n))$

⑤ little omega:- it is faster growth rate.

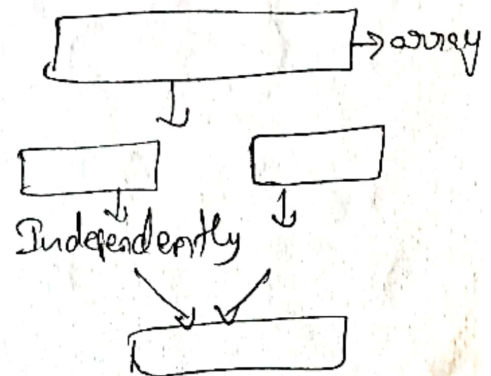
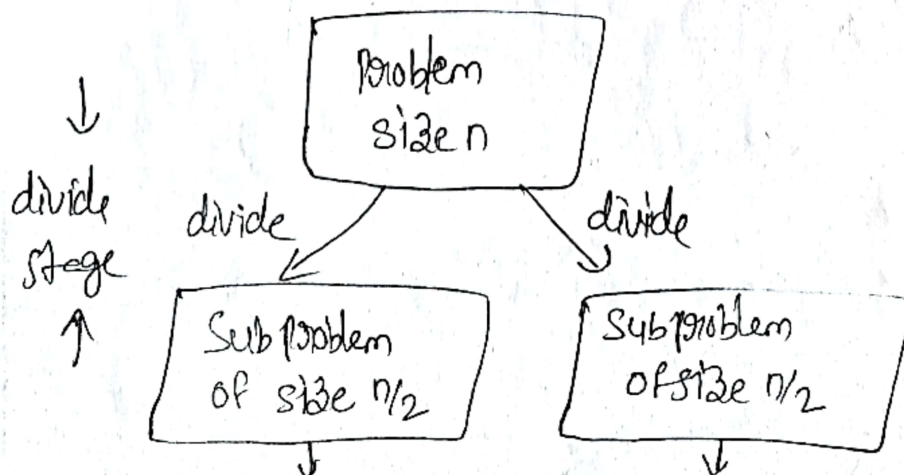
$f(n)$ grows faster than $g(n)$

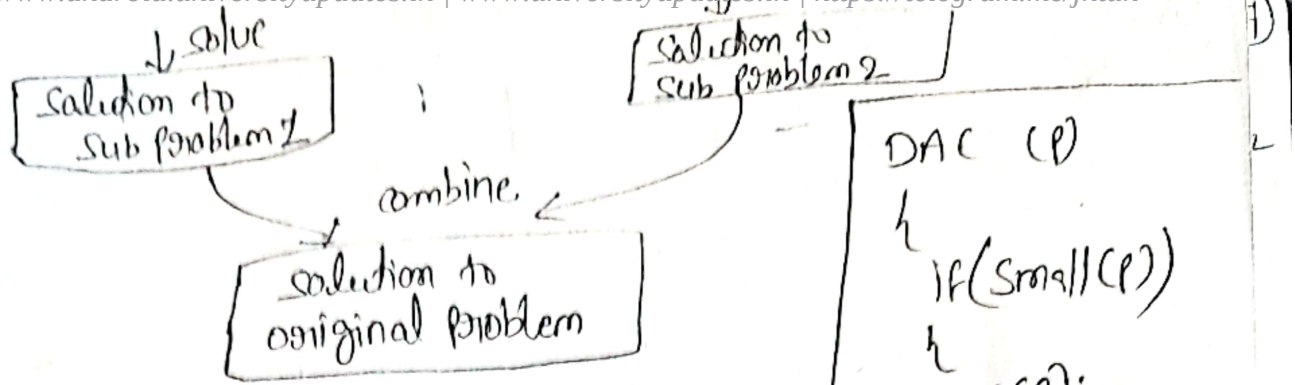
$$\text{if } \lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = \infty$$

formally stated as $f(n) = \omega(g(n))$

Divide and conquer:-

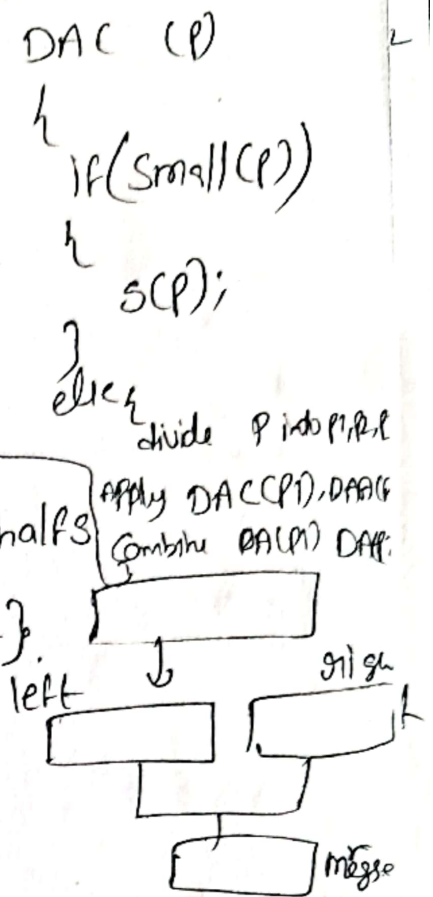
- ① Divide problem into smaller parts.
- ② Independently solve the parts.
- ③ Combine these solutions to get overall solution.





General methods:-

Idea 1:- Divide the array into 2 halves recursively sort left & right halves then merge two halves {merge sort}.



Idea 2:- Partition array into small items and large items then recursively sort the two sets {Quick sort}.

Examples for divide & conquer methods:-

- ① Searching : binary search
- ② sorting : merge sort, quick sort
- ③ tree traversals
- ④ matrix multiplication : Strassen's algorithm

Binary search:- It is an efficient algorithm for searching

in a sorted array.

Algorithm:-

- to look into something closely.
- Sorted array
- 
1. Inspect the middle item of an array of size n .
2. Inspect the middle item of an array of size $n/2$.
3. Inspect the middle item of an array of size $n/4$.
- & so on until lower bound becomes $>$ upper bound.

$\Rightarrow$ This implies $K = \log_2 N \rightarrow$ size of array.
 $\hookrightarrow$ No. of partitions.

Pseudo code:- it is nothing but it is a description of what an algorithm must do.

Pseudo code for binary search:-

Binary search ($A, \text{start}, \text{end}, \text{key}$)

```
{
  if ( $\text{start} \leq \text{end}$ )
  {
     $\text{mid} = \frac{(\text{start} + \text{end})}{2}$  // compute the mid element
    if (key equal to  $A[\text{mid}]$ )
    {
      return mid. // found it, successful search.
    }
    else if ( $\text{key} < A[\text{mid}]$ )
```

then return $\text{binarySearch}(\text{start}, \text{mid}-1, \text{key})$ // recursive called itself for lower part of an array

else if key
return binarySearch(mid+1, end, key) // recursively call itself
for upper part of an array.

}

return -1 // Unsuccessful search.

Example:- (i) successful search.

Let us take array

0	1	2	3	4	5	6	7	8	9
5	7	9	13	32	33	42	54	56	88

← shift →

∴ binary search can search the element in only sorted array.

key = 33

$$\text{mid} = \frac{(\text{start} + \text{end})}{2} = \frac{0 + 9}{2} = \frac{9}{2} \approx 4.$$

$33 > A[\text{mid}]$

$33 > 32$ ∴ start = mid + 1

5	6	7	8	9
33	42	54	56	88

$$\text{mid} = \frac{5 + 9}{2} = \frac{14}{2} = 7$$

$33 < A[\text{mid}]$

$33 < A[7]$ ∴ end = mid - 1

5	6
33	42

$$\text{mid} = \frac{5 + 6}{2} = \frac{11}{2} = 5.$$

mid == key

$33 = A[5]$

33 is found at index 5. then
it is successful search.

Binary Search (unsuccessful search):

0	1	2	3	4	5	6	7	8	9
5	7	9	13	32	33	42	54	56	88

key = 8 skip

$$\text{mid} = \frac{0+9}{2} = \frac{9}{2} \approx 4.$$

$$a[4] == a[\text{key}]$$

$$32 == 8 \quad (\text{false})$$

$$8 < a[\text{mid}]$$

$$8 < a[4]$$

$$8 < 32$$

$$\text{start} = \text{mid} + 1$$

5	7	9	13
0	1	2	3

skip

$$\text{mid} = \frac{0+3}{2} = \frac{3}{2} \approx 1$$

$$8 > a[\text{mid}]$$

$$8 > 7 \quad \text{start} = \text{mid} + 1$$

9	13
---	----

$$\text{mid} = \frac{2+3}{2} = \frac{5}{2} \approx 2$$

$$8 < a[\text{mid}]$$

$$8 < 9$$

$$\text{change} = \text{end} = \text{mid} - 1$$

So now

Start = 2 and end = 1

Start > end

it indicates 8 is not found in array.

This is an unsuccessful search.

Merge Sort :-

Basic plan

→ C Program to implement binary search :-

#include <stdio.h>

main()

{ int n, a[10], ~~key~~ low, mid, high, flag;

printf("enter array size");

scanf("%d", &n);

printf("enter array elements:");

for (i=0; i<n; i++)

scanf("%d", &a[i]);

printf("enter key element");

scanf("%d", &key);

flag = 0;

low = 0;

high = n-1;

```

while (low <= high)
{
    mid =  $\frac{low+high}{2}$ ;
    if (key == a[mid])
    {
        flag = 1;
        break;
    }
    else if (key < a[mid])
        high = mid - 1;
    else
        low = mid + 1;
}

```

```

if (flag == 1)

```

```

    printf("key element found");

```

```

else

```

```

    printf("key element not found");

```

```

}

```

Time complexity of binary search:-

① Best case Time Complexity:- $T(n) = O(1)$

② worst case Time Complexity:-

$T(n) = T\left(\frac{n}{2}\right) + 1$ $\rightarrow$ divide array into two parts
 $\rightarrow$ comparison

$\rightarrow$ Time Complexity of array of n elements

$$T(n) = T\left(\frac{n}{2}\right) + 1 \quad \Rightarrow T\left(\frac{n}{2^k}\right) + k \quad \therefore k=1$$

$$= T\left(\frac{n}{2}\right) + 1 + 1 \quad \Rightarrow T\left(\frac{n}{2^2}\right) + 2$$

$$= T\left(\frac{n}{4}\right) + 1 + 1$$

$$= T\left(\frac{n}{8}\right) + 1 + 1 + 1 \quad \Rightarrow T\left(\frac{n}{2^3}\right) + 3$$

$$= T\left(\frac{n}{8}\right) + 1 + 1 + 1$$

$$T(n) = T\left(\frac{n}{2^k}\right) + k$$

$$\text{let } 2^k = n$$

apply log on both sides

$$\log_2 2^k = \log n$$

$$k \log_2 2 = \log n$$

by default base is 2

$$k \log_2 2 = \log n \quad [\log_2 2 = 1]$$

$$k \cdot 1 = \log n$$

$$k = \log n$$

$$T(n) = T\left(\frac{2^k}{2^k}\right) + \log n \quad (\text{or}) \quad T\left(\frac{n}{n}\right) + \log n$$

$$= T(1) + \log n$$

$$= 1 + \log n$$

$$= \log n$$

Worst case Time complexity is $O(\log n)$;

5. Average case Time complexity:-

Average case Time complexity is $O(\log n)$

Quick sort:-

0	1	2	3	4	5	6	7
50	30	10	90	80	20	40	70
i							j

Pivot = 50

→ i is at 0th place, j is at 7th place

divide the array into two parts, ~~for~~ such that the left part contains the elements which are less than the pivot.

right part contains the elements which are greater than the pivot.

0	1	2	3	4	5	6	7
50	30	10	90	80	20	40	70
i							j
Pivot							

$50 < 50 \rightarrow$ it is true, so increment i

0	1	2	3	4	5	6	7
50	30	10	90	80	20	40	70
i							j

$30 < 50 \rightarrow$ it is true, so increment i

$10 < 50 \rightarrow$ " " "

$90 < 50 \rightarrow$ it is false.

Now j, $70 > 50 \rightarrow$ it is true, decrement j

$40 > 50 \rightarrow$ it is false

	i		j				
0	1	2	3	4	5	6	7
50	30	10	90	80	20	40	70
↓ pivot							

Now, we have check whether $i < j$
 $3 < 6 \rightarrow$ true

Swap 90 & 40, mean swap i & j

	i		j				
0	1	2	3	4	5	6	7
50	30	10	40	80	20	90	70
↓ pivot							

$40 < 50$; true increment i

$80 < 50$; false.

Now, j $\rightarrow 90 > 50 \rightarrow$ true, decrement j
 $20 > 50 \rightarrow$ false.

check $i < j \rightarrow$ true $4 < 5$

	i		j				
0	1	2	3	4	5	6	7
50	30	10	40	20	80	90	70
↓ pivot							

$20 < 50 \rightarrow$ condition is true, increment i value

$80 < 50 \rightarrow$ " is false

Now j, $80 > 50$, condition is true, decrement j value

$20 > 50$, " " false

$i < j \rightarrow 5 < 4$ condition is false

Swap pivot element with $a[j]$.

Swap(pival, a[j])

0	1	2	3	4	5	6	7
20	30	10	40	50	80	90	70

↑ pival

1	1	2	3	4
---	---	---	---	---

20 30 10 40 50
↑ pival

20 < 20 → true, increment i

30 < 20 → false

Now j, 50 > 20 → true, decrement j

40 > 20 → " " "

10 > 20 → false

So i < j, 1 < 2 true

Swap the element

0	1	2	3	4
---	---	---	---	---

20 10 30 40 50
↑ pival

10 < 20 → true, increment i

30 < 20 → false

Now j, 30 > 20 → true decrement j value

10 > 20 → false

check i < j, 2 < 1 → condition is false

Swap(pival, a[j]);

0 1 2 3 4
10 20 30 40 50
pivot
right part $\rightarrow$ 5 6 7
80 90 70
pivot

$80 < 90 \rightarrow \text{True}$, increment i

$90 < 80 \rightarrow \text{False}$

$70 > 80 \rightarrow \text{False}$

$i < j \Rightarrow 5 < 6$, swap i & j

70 80 90

80 70 90
pivot

$70 < 80 \rightarrow \text{True}$, increment i

$90 < 80 \rightarrow \text{False}$

$90 > 80 \rightarrow \text{True}$, decrement j

$70 > 80 \rightarrow \text{False}$

$i < j \rightarrow 7 < 6 \rightarrow \text{Condition is False}$

Now swap pivot element with j

5 6 7
70 80 90

0	1	2	3	4	5	6	7
10	20	30	40	50	70	80	90

Quick sort program:- and algorithm:-

Quick sort works based on the principle of Divide & Conquer algorithm.

1. Divide 2) conquer 3) combine

- First select the pivot element.
- Given the array into two parts. the left part contains the elements which are less than the element, & the right part contains the elements which are greater than the ^{pivot} element.
- In order to select pivot element, we have different strategies
 - 1) select first element as a pivot element.
 - 2) " last " " " " "
 - 3) select a random element from one of these elements.

```

eg:- #include <stdio.h>
      void main()
      {
        int low, high, i, j, pivot, temp; n,
        printf("read the size of array");
        scanf("%d", &n);
        printf("read the element");
        for(i=0; i<n; i++)
        {
          scanf("%d", &a[i]);
        }
        void quick_sort(int low, int high);
        {
          int p;
          if(low < high)

```

P = partition (low, high);

quick_sort (low, P-1);

quick_sort (P+1, high);

}

}

int partition (int low, int high)

{

int i, j, pivot, temp;

pivot = a[low];

i = low;

j = high;

while (i < j)

{

while (a[i] <= pivot)

i++;

while (a[j] > pivot)

j--;

if (i < j)

{

temp = a[i];

a[i] = a[j];

a[j] = temp;

} swap(a[i], a[j])

}

}

temp = a[low];

a[low] = a[j];

a[j] = temp

} swap(a[low], a[j])

return j;

}

Analysis of Quick sort:-

$$T(n) = 1 \rightarrow \text{element}.$$

→ Time complexity for array of n elements

$$T(n) = 2T\left(\frac{n}{2}\right) + n \rightarrow \text{minimum 'n' comparisons} \Rightarrow 2^1 T\left(\frac{n}{2^1}\right) + 1 \cdot n$$

$$= 2 \left[2T\left(\frac{n}{2}\right) + \frac{n}{2} \right] + n$$

$$= 4T\left(\frac{n}{4}\right) + \frac{2n}{2} + n$$

$$= 4T\left(\frac{n}{4}\right) + 2n \Rightarrow 2^2 T\left(\frac{n}{2^2}\right) + 2n$$

$$= 4 \left[2T\left(\frac{n}{2}\right) + \frac{n}{2} \right] + 2n$$

$$= 8T\left(\frac{n}{8}\right) + 3n$$

$$= 2^3 \cdot T\left(\frac{n}{2^3}\right) + 3n.$$

$$T(n) = 2^k \cdot T\left(\frac{n}{2^k}\right) + k \cdot n$$

$$\text{Let } 2^k = n$$

apply log on both sides

$$\log 2^k = \log n$$

$$k \log 2 = \log n$$

$$k \log k = \log n$$

$$k \cdot 1 = \log n$$

$$k = \log n$$

$$T(n) = n \cdot T\left(\frac{n}{n}\right) + \log n \cdot n$$

$$= n \cdot T(1) + n \log n$$

$$= n \cdot 1 + n \log n$$

$$= n \log n$$

The Best Case Time complexity is $O(n \log n)$.

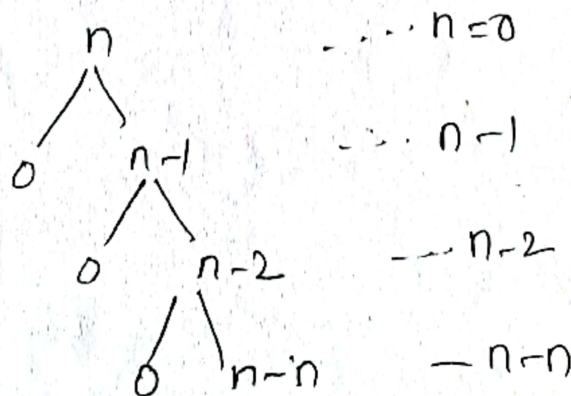
The best case & average case time complexity is 

$$O(n \log n)$$

Worst case

T.C:-

0 1 2 3 4
20 30 40 50 60



$$1+2+3+4+\dots = \frac{n(n+1)}{2} = \frac{n^2+n}{2}$$

$$O(n^2)$$

$$\log 2^k = \log n$$

$$k \cdot \log 2 = \log n$$

$$k \cdot 1 = \log n$$

Substitute this value

$$T(n) = n \cdot T\left(\frac{n}{n}\right) + \log n \cdot n$$

$$= n \cdot T(1) + n \log n$$

$$= n \cdot 1 + n \log n$$

$$= n + n \log n$$

$$= O(n \log n)$$

Strassen's matrix multiplication :-

Size $n \times n$

$$\text{eg: } \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} * \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix} = \begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{bmatrix}$$

$$C_{11} = A_{11}B_{11} + A_{12}B_{21}$$

$$C_{12} = A_{11}B_{12} + A_{12}B_{22}$$

$$C_{21} = A_{21}B_{11} + A_{22}B_{21}$$

$$C_{22} = A_{21}B_{12} + A_{22}B_{22}$$

• 2×2 matrix multiplication can be accomplished in 8 mod.
min multiplication.

$$2^{\log_2 8} = 2^3$$

$$2^{\log_2 2^3} \geq 2^3$$

Basic algorithm

void matrix_mul (int a[][], int b[][], int c[][])

{

for (i=1; i<=n; i++) $\rightarrow n+1$

{

for (j=1; j<=n; j++) $\rightarrow n \times (n+1)$

{

c[i][j] = 0; $\rightarrow 1 \times 1 \times 1$

for (int k=1; k<=n; k++) $\rightarrow n \times n \times (n+1)$

{

c[i][j] = c[i][j] + [A[i][k] * B[k][j]]; $\rightarrow 1 \times 1 \times 1$

}

}

}

$$\begin{matrix} n+1 \\ n^2+n \\ 1 \\ n^3+n^2 \\ 1 \end{matrix}$$

$$\Rightarrow n^3 + 2n^2 + 2n + 3$$

$$\Rightarrow \boxed{O(n^3)}$$

Time analysis:-

$$C_{ij} = \sum_{k=1}^N a_{ik} b_{kj}$$

$$\text{Thus } T(N) = \sum_{i=1}^N \sum_{j=1}^N \sum_{k=1}^N C$$

$$= C N^3$$

$$= O(N^3)$$

P, Q, R, S, T, U, V can be computed using 7 matrix multiplication and 10 matrix additions or subtractions. (17)

$$P = (A_{11} + A_{22})(B_{11} + B_{22})$$

$$Q = (A_{21} + A_{22})B_{11}$$

$$R = A_{11}(B_{12} - B_{22})$$

$$S = A_{22}(B_{21} - B_{11})$$

$$T = (A_{11} + A_{12})B_{22}$$

$$U = (A_{21} - A_{11})(B_{11} + B_{12})$$

$$V = (A_{12} - A_{22})(B_{21} + B_{22})$$

$$C_{11} = P + S - T + V$$

$$C_{12} = R + T$$

$$C_{21} = Q + S$$

$$C_{22} = P + R - Q + U$$

$$T(n) = O(n^{\log_2 7})$$

$$= \underline{\underline{O(n^{2.81})}}$$